

GRADUATION PROJECT PROGRESS REPORT

Automated Post-Hospital Patient Monitoring System

A Scalable, Cloud-Enabled Data Engineering Ingestion & Quality Pipeline

Course: DEPI Graduation Project

Prepared By: Graduation Project Team

Project Domain: Data Engineering

Submitted To: University Project Supervisor

Submission Date: June 30, 2026

Department: Computer & Data Engineering

Table of Contents

Section	Page
1. Project Overview & Context	2
2. Problem Statement & Project Objectives	2
3. Technology Stack & Architecture	3
4. Dataset Description & Profiling	3
5. Data Quality Assessment (Audit)	4
6. Data Cleaning & Transformation Pipeline	4
7. Implemented Code & Reports Registry	5
8. Project Status, Completion Percentage & Justification	5
9. Risks, Limitations & Next Development Phase	6
10. Conclusion	6

1. Project Overview & Context

The "Automated Post-Hospital Patient Monitoring System" is a dedicated Data Engineering graduation project designed to build a robust, scalable, and automated telemetry ingestion and processing platform. As patients are discharged from medical centers, their continuous vital signs are collected via clinical monitoring devices. This project establishes a data pipeline that ingests this high-frequency vital signs stream, subjects it to data quality assurance processes, and loads it into a high-performance database. The primary target is to enable real-time dashboarding, alert triggering, and historical analysis. Importantly, this project focusing strictly on infrastructure, orchestration, and database design, and is completely free of Machine Learning components.

Why this project was chosen:

Post-hospitalization patient monitoring represents an ideal intersection between vital healthcare telemetry and cloud data architecture. By designing and building this pipeline, we demonstrate capability in managing complex data ingestion lifecycles (ingestion, staging, validation, transformations, and load layers) while ensuring data privacy, security, and strict data quality gates. In a live setting, such pipelines provide immediate, actionable insight to doctors, drastically reducing patient readmission rates.

2. Problem Statement & Project Objectives

Problem Statement: Once a patient leaves the medical facility, continuous care is interrupted. Early clinical deterioration is frequently missed due to sporadic or unstandardized telemetry collection. To construct a reliable monitoring dashboard, clinical analysts require a high-fidelity, clean database of vitals. However, raw IoT vital logs suffer from bad data shapes, unit mismatches, out-of-sync timestamps, and redundant mathematical indicators. Without an automated pipeline that applies rigorous data quality gates and standardized naming conventions, clinical alert systems cannot operate effectively.

Project Objectives:

- Build a professional, modular folder structure suitable for automated pipeline deployments.
- Set up a standardized virtual environment to isolate library dependencies.
- Perform a thorough, code-driven profiling study of the raw clinical vital signs dataset.
- Perform a data quality assessment to define the target relational production schema.
- Create a reusable, automated Python data cleaning pipeline to structure and parse inputs.
- Deploy Azure resources (ADF, Blob Storage, SQL Database) to schedule and run the pipeline (Planned).

3. Technology Stack & Rationale

Component	Technology	Rationale & Selection Justification
Processing Core	Python 3.12 (Pandas)	Industry-standard for data parsing and cleaning. Pandas provides high-performance column manipulations.
Storage / Database	Azure SQL Database	Enterprise-grade relational storage with native connection pooling, clustering index supports, and security integration.
Orchestration	Azure Data Factory (ADF)	Cloud scheduling and workflow coordination. Triggers python cleaning scripts and SQL loading tasks.
Staging Area	Azure Blob Storage	Cost-effective object storage to host the landing raw CSV logs and clean intermediate files.
Dev & Control	VS Code / Git	Standardized IDE environment and secure version control utilizing environment configurations.

4. Current System Architecture

The system pipeline follows an ingestion, cleaning, staging, and relational loading architecture. Below is the workflow diagram:

Phase	Source / System	Operations / Tasks
1. Raw Ingestion	Azure Blob (Raw Container)	Incoming telemetry CSV files land in the cloud storage bucket.
2. Cleaning & Staging	Python script in ADF / VM	Executes data cleaning, drops derived columns, renames to snake_case, and exports clean CSV.
3. Relational Load	Azure SQL Database (patient_vitals)	Loads processed records into database. Enabled dynamic query views for dashboarding (Planned).

5. Dataset Description & Profiling

Dataset Profile (Raw): The dataset ``human_vital_signs_dataset_2024.csv`` was analyzed programmatically. It contains exactly **200,020 rows** and **17 columns**. Profiling verified that the raw dataset has **0% missing values** across all columns and contains **0 duplicate rows**.

Key statistics extracted from numerical features during profiling:

Feature Column	Mean Value	Min	Max	Clinical Interpretation
Heart Rate	79.53 bpm	60	99	Ranges between standard resting limits.
Oxygen Saturation	97.50%	95.0%	100.0 %	Normal physiological range (95%+).
Systolic BP	124.44 mmHg	110	139	Pre-hypertension to standard blood pressure.
Diastolic BP	79.50 mmHg	70	89	Normal diastolic range.
Age	53.45 years	18	89	Broad patient population coverage.
Weight	75.00 kg	50.0	100.0	Standard weight profile.
Height	1.75 meters	1.5	2.0	Standard height profile.

6. Data Quality Assessment (Audit Recommendations)

A formal data quality audit was conducted on the raw data columns. The audit highlighted major structural redundancies in the derived metrics. The recommendations approved by the supervisor are as follows:

Raw Column	Status / Action	Architectural Justification
Patient ID	Keep & Rename	Remains as primary identifier. Set as Primary Key (PK).
Timestamp	Keep, Parse & Cast	Must be converted from string/text type to proper Datetime object for timeseries query partitions.
Derived_BMI	Remove	Redundant. Directly computed via $\text{Weight} / \text{Height}^2$. Storing calculated columns in relational tables violates 3NF normalization rules and leads to database anomalies.
Derived_MAP	Remove	Redundant. Calculated mathematically from Systolic and Diastolic Blood Pressures.
Derived_Pulse_Pressure	Remove	Redundant. Calculated dynamically as $\text{Systolic} - \text{Diastolic}$.
Derived_HRV	Keep & Rename (hrv)	Heart Rate Variability is a complex sensor measurement, not easily reverse-engineered. Renamed to simple snake_case.
Risk Category	Keep & Rename (risk_category)	Retained in order to compare model/alarm classifications later.

7. Data Cleaning & Transformation Pipeline

A modular, automated Python cleaning script was built under `src/cleaning/clean_data.py` to automate these changes. Upon loading the raw CSV data into memory, the script performs the following sequential actions:

- Drop derived features:** Dropped the columns `Derived_BMI`, `Derived_MAP`, and `Derived_Pulse_Pressure`.
- Apply Naming Conventions:** Re-mapped and renamed remaining mixed-case and bracketed columns into lowercase `snake_case` (e.g. `Patient ID` $\rightarrow$ `patient_id`, `Weight (kg)` $\rightarrow$ `weight_kg`).
- Convert Timestamps:** Converted the string `Timestamp` column to proper Python/Pandas `datetime64[ns]` type.
- Export Clean Output:** Saved the resulting database-ready data matrix (containing 200,020 rows and 14 columns) to `data/processed/patient_vitals_clean.csv`.
- Write Report:** Generated a data cleaning execution summary logged directly to `docs/data_cleaning_report.md`.

8. Implemented Code & Reports Registry

The table below details every workspace file and report successfully created, along with its function and contribution:

Workspace File / Path	Type	Purpose & Contribution to Pipeline
`src/cleaning/clean_data.py`	Python Module	Consists of `clean_dataset()` and `generate_cleaning_report()` which automates parsing, drops columns, renames to snake_case, and exports output.
`docs/data_profile.md`	Markdown Doc	Formulates the initial data profile. Provides records counts, missing value analysis, duplicates, and key statistics.
`docs/data_quality_assessment.md`	Markdown Doc	Pre-ETL quality audit outlining normalisation suggestions, calculated views, and Azure SQL schema recommendations.
`docs/data_cleaning_report.md`	Markdown Doc	Summary report verifying column counts, memory footprint, and data types before/after the cleaning runs.
`data/processed/patient_vitals_clean.csv`	Dataset File	Final cleaned data output, structured in snake_case and ready for database insertion.

9. Project Status, Completion Percentage & Justification

Current Completion Percentage: 35%

Justification:

- **Phase 1: Project Setup (100% completed):** Root folders, virtual environment, .gitignore, .env.example, and libraries are complete.
- **Phase 2: Ingestion & Data Profile (100% completed):** Raw files moved, profiling script executed, and stats verified.
- **Phase 3: Data Quality & Audit (100% completed):** Comprehensive schema assessment and data audit completed.
- **Phase 4: Data Cleaning Pipeline (100% completed):** Automated Python module implemented, run, and verified.
- **Phase 5: Azure Cloud Deployment & Storage (Planned - 0% completed):** Azure Blob Storage, ADF triggers, connection strings.
- **Phase 6: Relational Database & SQL (Planned - 0% completed):** SQL DDL schema deployment, index tuning, and calculated views.
- **Phase 7: End-to-End Orchestration & Testing (Planned - 0% completed):** Automating pipeline runs under ADF schedule.

10. Risks, Limitations & Next Development Phase

Risks and Limitations:

- **Data Volume Scaling:** The cleaning pipeline currently runs locally using Pandas which loads the dataset entirely into memory. While sufficient for 200k rows (38MB), it will experience performance degradation if patient telemetry grows into gigabytes. Transitioning to PySpark or Azure Synapse Spark pools will be required for high-volume settings.
- **Connection and Security:** Moving variables to Azure will require Key Vault configurations. Hardcoding connection keys must be strictly prohibited, which we have mitigated using the ``.env.example`` configurations.
- **Timestamp Precision:** Telemetry uses microsecond logging. If database indexes are not properly configured, time-window aggregations will experience severe latency.

Next Development Phase:

Our next step is the database design and cloud configuration. We will define and write standard SQL deployment scripts under ``sql/`` to compile the single production table ``patient_vitals``. Following this, we will transition to Azure cloud setup, deploying Azure Storage Blob, Azure SQL Database, and defining ADF Copy activities to link the ingestion pipelines.

11. Conclusion

The "Automated Post-Hospital Patient Monitoring System" has completed its foundational pre-ingestion phases. The raw telemetry has been profiled, structures audited, and a robust cleaning script executed. The clean dataset, saved in a database-compatible snake_case form, is staged at ``data/processed/patient_vitals_clean.csv``. All actions have been formally documented in profiling, quality, and cleaning reports, establishing a production-ready starting point for the subsequent database DDL integration and Azure orchestration phases.

Submitted By:

Approved By:

Graduation Project Team

Project Supervisor